

PROJECT DOCUMENTATION

NeuroScope

Architecture & Folder Structure Guide



Team: DigiNeurons

Version: 0.1.0

Date: June 2026

Project Overview

NeuroScope is a full-stack application with a **Python backend** (FastAPI) and a **React frontend** (Three.js). The backend parses neural network models and runs analysis. The frontend renders interactive 3D visualizations.

How It Works

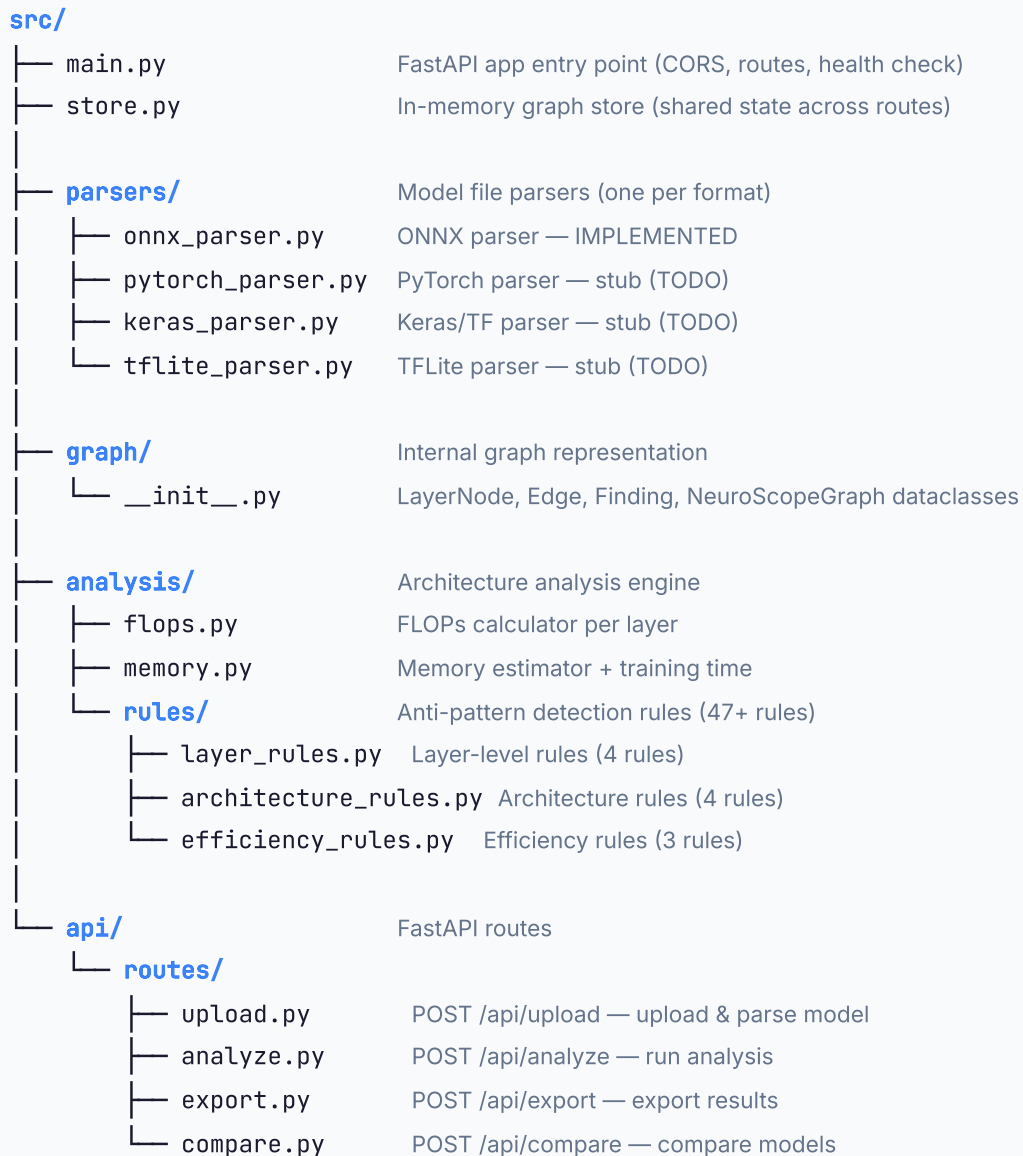


Top-Level Structure

neuroscope/	
├─ src/	Python backend (FastAPI)
├─ frontend/	React frontend (Three.js)
├─ config/	Configuration files
├─ tests/	Test suite
├─ docker/	Dockerfiles
├─ docs/	Documentation
├─ research/	Technical research
├─ competition/	AYAIR 2026 materials
├─ scripts/	Utility scripts
├─ README.md	Project overview
├─ requirements.txt	Python dependencies
├─ docker-compose.yml	Docker orchestration
└─ LICENSE	MIT License

Backend — src/

The backend is a **FastAPI** application. It handles model parsing, architecture analysis, and serves the API endpoints that the frontend consumes.



Key Backend Files

src/main.py

The FastAPI application. Sets up CORS, rate limiting, health check, and registers all API route modules. This is what `uvicorn src.main:app` runs.

src/store.py

Thread-safe in-memory store. When a model is uploaded and parsed, the graph is stored here with a UUID key. The analyze and export routes look up graphs by this UUID.

src/graph/__init__.py

Defines the core data structures: **LayerNode** (single layer), **Edge** (connection), **Finding** (analysis result), **NeuroScopeGraph** (complete model). All parsers produce this format.

Frontend — frontend/

The frontend is a **React + TypeScript** application with **Three.js** for 3D rendering. Built with **Vite** for fast development.

frontend/

├─ package.json	Dependencies (React, Three.js, Zustand, Axios)
├─ vite.config.ts	Vite build config + API proxy
├─ tsconfig.json	TypeScript configuration
├─ index.html	HTML entry point
└─	
├─ src/	
│ └─ main.tsx	React entry point — mounts App
│ └─ App.tsx	Main layout — upload view or workspace view
│ └─ store.ts	Zustand state (graphData, analysisData, selectedLayer)
│ └─ types.ts	TypeScript interfaces (LayerNode, Edge, GraphData, etc.)
│ └─ index.css	Global styles (dark theme)
│ └─	
│ └─ components/	
│ └─ UploadZone.tsx	Drag-and-drop file upload with progress bar
│ └─ Canvas3D.tsx	Three.js 3D visualization of the model
│ └─ AnalysisPanel.tsx	Shows analysis findings (warnings/errors)
│ └─ StatsPanel.tsx	FLOPs, memory, params overview
│ └─ LayerDetail.tsx	Click a layer → see description + code
│ └─ ExportMenu.tsx	Export as JSON, TXT, or PNG

Component Flow



Configuration — config/

config/

— analysis_rules.yaml	All 11 anti-pattern rules with thresholds
— layer_shapes.yaml	Layer type → 3D shape mapping
— languages/	
— en.json	English translations

analysis_rules.yaml

Configures severity levels (CRITICAL/WARNING/INFO), thresholds (e.g., min_depth=15 for skip connections), and enable/disable toggles. Also defines hardware presets for training time estimation (T4, V100, A100, RTX3090, RTX4090).

layer_shapes.yaml

Maps each layer category to a 3D geometry: convolution→box, activation→sphere, recurrent→cylinder, attention→octahedron, pooling→small_cube, normalization→slab, etc.

Docker & Tests

docker/

docker/

└─ Dockerfile.backend	Python backend container (FastAPI + ONNX)
└─ Dockerfile.frontend	React frontend container (nginx)

docker-compose.yml

Orchestrates both containers. Backend on port 8000, frontend on port 3000. Includes health checks, resource limits, and volume mounts for data and config.

tests/

tests/

└─ test_parsers/	
└─ test_onnx_parser.py	ONNX parser unit tests
└─ test_analysis/	
└─ test_flops.py	FLOPs calculation tests
└─ test_rules.py	Analysis rules tests
└─ test_graph/	
└─ test_graph.py	Graph data structure tests

Documentation & Competition

docs/

docs/

— week1/	Week 1 report (HTML + PDF)
— week2/	Week 2 report (HTML + PDF)
— week3/	Week 3 report (HTML + PDF)
— week4/	Week 4 report (HTML + PDF)
— specs/	Technical specifications

competition/

competition/

— <code>essay_final.md</code>	Project introduction essay (800-word limit)
— <code>team_roster.md</code>	9 team members with roles
— <code>submission_checklist.md</code>	AYAIR 2026 submission steps
— <code>registration.md</code>	Registration form template
— <code>timeline.md</code>	Development timeline
— <code>*.pdf</code>	Generated PDFs (comprehensive + essay)

research/

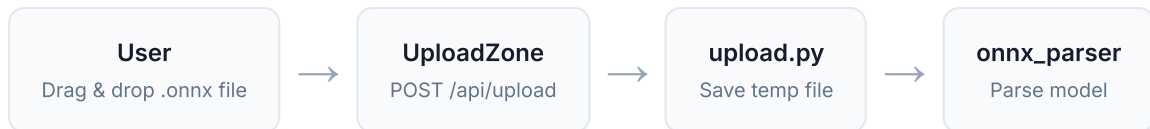
research/

— <code>competitor_analysis.md</code>	15 competing tools analyzed
— <code>african_ml_landscape.md</code>	ML education in Africa
— <code>ml_anti_patterns.md</code>	Common deep learning mistakes
— <code>tech_stack.md</code>	Technology decisions

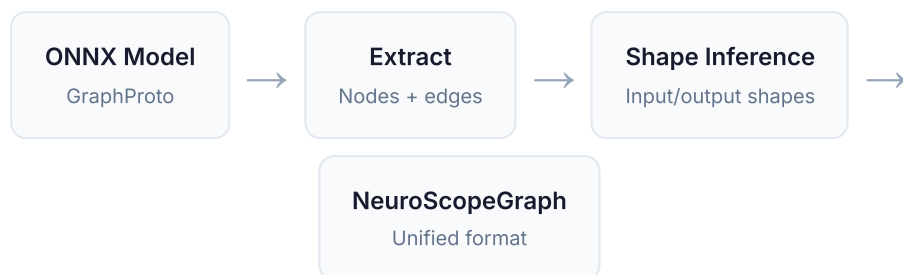
Complete Data Flow

Here's exactly what happens when a user uploads a model:

Step 1: Upload



Step 2: Parse



Step 3: Analyze



Step 4: Visualize

